```
var tokenId = tokenStore.create(request, token);
var base = URI.create(request.url());
try {
return new URI(base.getScheme(), ":" + tokenId, #A
base.getHost(), #A
base.getPort(), path, null, null); #A
} catch (URISyntaxException e) {
throw new RuntimeException(e);
}
}
public void lookupPermissions(Request request, Response
response) {
var authHeader = request.headers("Authorization"); #B
if (authHeader == null || !authHeader.startsWith("Bearer "))
#B
return; #B
var tokenId = authHeader.substring(7).trim(); #B
tokenStore.read(request, tokenId).ifPresent(token -> {
var tokenPath = token.attributes.get("path");
if (Objects.equals(tokenPath, request.pathInfo())) {
request.attribute("perms",
token.attributes.get("perms"));
}
});
}
```

#A Construct a new URI passing in the token as the userinfo password component.
#B Expect the token in the Authorization header.

Due to the insecurities of including user credentials directly in a URI and the potential for phishing, browsers now aggressively block this form of URI or silently remove the credentials when following such a link. This makes the userinfo component a poor choice for storing the token when using capability URIs directly in a browser, but for non-